Cost-Sensitive Predictive Maintenance for Turbofan Engines: Learning Remaining Useful Life for Operational Decision Support

Cole H. Shaigec, Kelly Falcón, Youwei Chen
Boston University

Abstract

This project studies predictive maintenance for turbofan engines using the NASA CMAPSS dataset. We compare classical machine learning models for remaining useful life estimation and failure warning, and evaluate them through an explicit maintenance cost model rather than prediction accuracy alone. Our results show that decision quality depends strongly on warning horizon, class balance, and the interaction between model outputs and decision boundaries under asymmetric costs. These findings motivate decision-aware evaluation and policy-based formulations for practical predictive maintenance systems.

1. Introduction

Airlines operate in economically competitive and safety-critical environments, where maintenance decisions must balance cost efficiency against the risk of failure. Industry reports suggest that turbofan engine maintenance constitutes a substantial portion of operating expenses [1]. However, the remaining useful life (RUL) of an engine is a latent state, forcing airlines to make maintenance decisions under uncertainty. Premature maintenance wastes useful component life and incurs unnecessary cost, while delayed maintenance risks catastrophic failure with severe safety and economic consequences. This tension motivates data-driven methods that estimate engine health from observed data and support effective maintenance decision-making.

Modern jet engines generate high-dimensional sensor data, enabling estimation of engine health states from observed behavior. This induces a supervised learning problem in which models are trained to predict engine failure risk or RUL from sensor data. Given the problem context, however, predictive accuracy alone is not sufficient: the ultimate objective is to facilitate economically and operationally sound maintenance decisions.

To study this problem, we use NASA's CMAPSS turbofan engine degradation dataset, which provides multivariate time-series sensor measurements and corresponding RUL labels under simulated operating conditions. We develop a modular machine learning pipeline and use it to train and systematically evaluate a set of classical machine learning models on the CMAPSS dataset. We then define a simplified economic model and use it to assess the performance of the maintenance policies induced by these models under a cost-sensitive decision framework.

2. Literature Review

Predictive maintenance for turbofan engines has been widely studied using the NASA CMAPSS datasets, which provide multivariate time-series sensor data for RUL estimation under varying operating conditions [2]. Prior work on industrial data quality highlights noise as a practical modeling issue [5]. Prior work has shown that data-driven approaches, including similarity-based methods, can effectively capture degradation patterns without requiring detailed physical models [3]. Classical machine learning methods remain competitive for structured, tabular data, offering interpretability and strong performance without the data and computational requirements of deep learning.

Most prior work evaluates predictive models using statistical metrics such as RMSE or classification accuracy. However, in operational settings, the ultimate objective is decision quality under asymmetric cost. This motivates a decision-aware perspective in which predictive models are assessed in terms of the maintenance policies they induce, rather than prediction accuracy alone, aligning with broader decision-analytic frameworks [6].

3. Problem Formulation and Solution Approaches

3.1. Problem Setup

We consider a predictive maintenance setting based on the NASA CMAPSS dataset. Given sensor observations $\mathbf{x}_i$ for engine i, we learn a predictor $f_\Theta(\mathbf{x}_i)$ that estimates RUL.

We study both regression and classification formulations. In the former, we compute the RUL estimate $\hat{r}_i$ directly. In the latter, we define a warning horizon T and

learn a binary label $\hat{y}_i := \mathrm{sign}(T - r_i)$. If the chosen T is near a tail of the underlying RUL distribution, large class imbalances may be induced by this mapping [4].

While prediction accuracy is important, our ultimate objective is decision quality under a maintenance policy derived from these predictions.

3.2. Dataset and Preprocessing

The CMAPSS dataset is divided into four subsets, FD001–FD004 [2]. We select FD001 and FD003 for our study. FD001 models a single physical failure mechanism, degradation of the high-performance compressor (HPC). FD003 models two degradation mechanisms, the HPC and the fan, and allows them to interact. We hypothesize that this heterogeneous data-generating process will make the learning problem more challenging for FD003.

We construct feature vectors from sliding windows of multivariate sensor observations. Features with near-zero variance are removed, and inputs are standardized. Optional dimensionality reduction via PCA is also explored [8]; empirically, PCA did not materially improve decision quality (Appendix B).

3.3. Models

We consider a set of classical machine learning models: ridge regression [7], quadratic discriminant analysis (QDA), Naive Bayes [8], and random forests [9]. This selection enables comparison across linear, probabilistic, and nonlinear modeling assumptions while maintaining computational tractability. We also design a weighted ridge variant to evaluate the impact of cost-sensitive weighting on decision quality. k-nearest neighbors (kNN) was initially considered but excluded from final experiments due to computational cost under the sliding-window feature construction (Appendix K).

3.4. Policy Induction and Cost Model

Predictions induce the maintenance policy $\pi_i = \mathbf{1}\{\hat{y}_i = +1\}$ for classification and $\pi_i = \mathbf{1}\{\hat{r}_i \leq T\}$ for regression. Maintenance is performed on engine i when $\pi_i = 1$.

Policy performance over warning horizon T is evaluated using the cost model:

$$C(\boldsymbol{\pi}, T) = \sum_{i=1}^{n} \big[C_M \pi_i + C_F (1 - \pi_i) \mathbf{1}\{r_i \leq T\} + \alpha \max(r_i - T, 0) \pi_i \big]. \tag{1}$$

C_M denotes maintenance cost, C_F failure cost, and α penalizes premature maintenance in proportion to wasted RUL. We assume $C_F \gg C_M$, reflecting asymmetric costs of failure versus early repair in industrial maintenance settings [1]. This induces an asymmetric loss landscape in which false negatives dominate decision quality, motivating recall-oriented tuning strategies and cost-aware evaluation [6].

3.5. Learning Problem

For a given warning horizon T, the learning problem is to find a parameterized predictor f_Θ that induces a cost-minimizing maintenance policy:

$$\Theta^* := \arg\min_{\Theta} C\big(\boldsymbol{\pi}(f_\Theta), T\big). \tag{2}$$

Because the policy cost is non-differentiable with respect to model parameters, we approximate this objective by training under standard statistical losses and tuning hyperparameters via policy cost. [6] (see Section 4.3).

3.6. Evaluation Criteria and Experimental Questions

While standard statistical metrics (e.g., RMSE and F1-score) are computed, policy cost is treated as the primary evaluation criterion. We investigate the following questions:

- Does cost-sensitive ridge weighting improve decision quality relative to standard ridge regression?
- Does regression-based RUL estimation provide superior decision quality under the cost model compared to threshold-based classification?
- How does warning horizon affect policy and economic performance?
- How strongly do traditional machine learning metrics reflect decision quality under asymmetric cost?
- How does physical failure-process complexity influence predictive performance and decision quality?

4. Implementation

Given our large search space, we constructed a modular and configurable pipeline to enable consistent, rapid experimentation. This pipeline is described in Appendix J.

4.1. Preprocessing

Data are partitioned at the engine level into cross-validation folds to prevent leakage across temporally correlated observations. Prior to cross-validation, feature selection is performed by removing sensors with near-zero variance. Within each fold, preprocessing transformations are fit on the training data and applied to both training and validation sets. These include z-score standardization and optional PCA. Sliding window transformations construct fixed-length feature vectors from time-series observations.

4.2. Experimental Configuration

Experiments are conducted over a configurable search space spanning model choice, warning horizon T, window

size, and preprocessing options (Appendix C). The pipeline systematically evaluates combinations from this space to generate a broad set of experimental configurations.

4.3. Training and Evaluation

Regression models are trained using standard statistical objectives and tuned with respect to policy cost, while classification models are tuned using recall as a proxy to prioritize failure detection under asymmetric cost. Hyperparameters are tuned via grid search and 5-fold cross-validation, with folds constructed at the engine level to prevent leakage across temporally correlated observations. Models are retrained on the full training data and evaluated on the held-out test set.

4.4. Model Implementation Choices

All models are implemented with standardized interfaces for training and prediction. Standard implementations (e.g., MATLAB's `fitcensemble` for random forests) are used where appropriate. Custom implementations are employed for certain models. The approaches used for QDA, weighted ridge regression, and Naive Bayes are described in Appendices D–F.

5. Results

We now present the results of our experiments, answering the aforementioned research questions in sequence.

5.1. Evaluation of Cost-Sensitive Ridge Regression Weighting Scheme

We test a cost-sensitive weighting scheme for ridge regression (Appendix E) and compare its performance with unweighted ridge regression across 800 matched experiment configurations. The results are visualized in Figure 1, and deeper statistical test results are presented in Appendix G.

Our statistical tests indicate that weighted ridge does not improve cost performance on average (Δcost ≈ 0, $p \approx 0.99$; ΔRMSE ≈ 0, $p \approx 0.98$). However, it consistently reduces policy cost in short-horizon regimes ($T < 10$), where near-failure decisions dominate. These results indicate that the weighting scheme does not improve global performance but instead acts locally near the decision boundary. In particular, weighting increases the influence of near-failure samples, effectively shifting the learned regression function in regions where threshold crossings determine policy actions. As a result, improvements occur only in short-horizon regimes, where small prediction shifts alter decisions. Outside these regimes, where decisions are insensitive to small perturbations in predicted RUL, the weighting provides no measurable benefit.

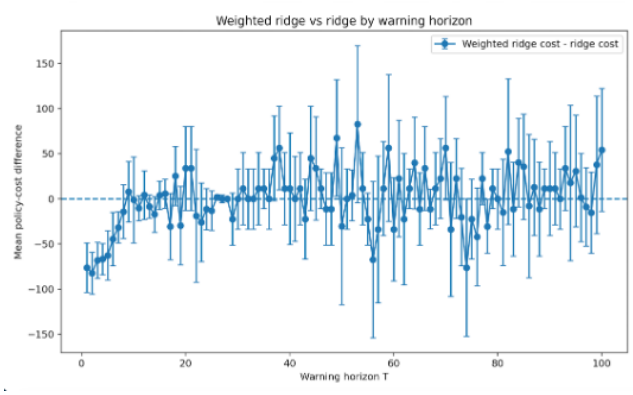

Figure 1. Weighted/unweighted ridge regression policy cost comparison across warning horizons.

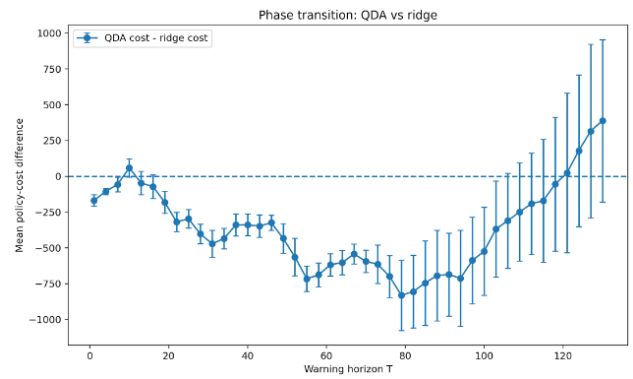

Figure 2. QDA/unweighted ridge policy cost comparison.

5.2. Comparison of Classification and Regression Models

To compare our classification and regression formulations, we first performed an initial sweep to identify the best-performing classification and regression models across a broad range of scenarios. QDA was the best-performing classifier, while unweighted and weighted ridge regression proved largely indistinguishable. We thus chose unweighted ridge as our regression representative for computational reasons and ran a comparison of QDA and unweighted ridge over various warning horizons. The results are shown in Figure 2.

Figure 2 shows that QDA consistently achieves lower policy cost than ridge across most warning horizons, with the largest gains in intermediate regimes. Ridge becomes competitive only at extreme horizons, where the decision problem becomes either trivial (very small T) or highly uncertain (very large T). One contributing factor is the class imbalance induced by the warning-horizon labeling: for extreme values of T, one class dominates, degrading classifier performance and reducing the advantage of direct classifi-

cation. This indicates that classification-based approaches are generally preferable, but their benefit depends on the induced class balance. This behavior reflects a structural difference between the formulations: classification models learn the decision boundary directly, whereas regression models must approximate it indirectly through a threshold. This indirection introduces inefficiency, as regression accuracy improvements only affect policy when they move predictions across the decision boundary.

5.3. Impact of Warning Horizon on Decision Quality

Our ridge/QDA comparison was extended to test the impact of warning horizon on decision quality. The results are shown in Figure 3.

Figure 3 shows that policy cost increases consistently in the warning horizon. This coincides with a corresponding increase in missed failures, which dominate the cost function. As the warning horizon grows, the decision boundary is pushed further from the failure point into regions where degradation signals are weaker and less separable, making accurate prediction more difficult. From a statistical perspective, increasing T moves the decision boundary into regions of lower signal-to-noise ratio, increasing model uncertainty and the likelihood of costly classification errors, particularly false negatives.

The operational implications of this result must be interpreted with care. We note as a limitation of our study that we have framed maintenance policy as a one-shot decision problem. In practice, however, maintenance decisions are taken sequentially, and postponing decisions until failure is imminent reduces scope to correct costly type II errors. Thus, the observed advantage of small warning horizons may not translate cleanly to real systems, where decision policy must balance one-shot cost performance with flexibility over time. A true "optimal" warning horizon cannot be defensibly determined from a one-shot framework.

5.4. Evaluation of Traditional Machine Learning Metrics as Decision Quality Proxies

We tested the performance of various traditional machine learning metrics as proxies for decision quality under the asymmetric cost structure inherent to our problem. The results are summarized in Figure 4.

All correlations in Figure 4 are statistically significant (Appendix H), but most effect sizes are small. Recall, a metric that penalizes costly false negatives, is a useful proxy for decision quality, while precision, which targets less costly false positives, is less valuable. Regression metrics like RMSE optimize for a fundamentally different objective than decision quality and thus exhibit weak relationships to policy cost. This is expected given our problem context: the large residuals that these metrics penalize are inconsequential for policy unless they push a sample across the decision boundary. This reflects a fundamental misalignment: traditional metrics optimize error magnitude, whereas decision quality depends only on whether prediction errors cross the policy threshold. Errors that do not alter decisions have no operational consequence, rendering many statistically significant improvements irrelevant from a decision perspective.

These results support direct policy cost as a first-class model evaluation objective for practical predictive maintenance problems: strong performance on traditional ML metrics is not ipso facto evidence of decision quality, and model selection based on predictive metrics alone is fundamentally misaligned with decision objectives under asymmetric cost.

5.5. Impact of Failure Mode Complexity on Decision Quality

To test our hypothesis that interaction of heterogeneous failure modes would complicate the learning problem, we compared model performance across CMAPSS subsets FD001 and FD003. The results, presented in Figure 5 and Appendix I, were unexpected.

All five models tested achieved lower policy cost on FD003, the subset with the more complex failure mechanism. These differences were statistically significant for all models except the random forest. This result challenges the assumption that greater physical failure mode complexity necessarily leads to a more difficult learning problem. Instead, it suggests that predictive performance depends less on underlying failure mode complexity and more on the separability of observable degradation signals in feature space. It also suggests quantification of separability (e.g., class overlap and margin structure) vis-à-vis decision performance as a promising avenue for future work.

5.6. Summary of Results

Taken together, these results show that decision quality in predictive maintenance is governed not by predictive accuracy alone, but by how model outputs interact with decision boundaries under asymmetric costs. Performance depends strongly on operating regime, class balance, and the separability of degradation signals. Standard ML metrics do not reliably capture these effects, and model improvements that do not alter decisions have no operational impact. These findings motivate a shift from prediction-centric to decision-aware model design and evaluation.

6. Conclusion and Implications for Practice

This study demonstrates that predictive maintenance performance is governed not by predictive accuracy in isolation, but by the interaction of model outputs and decision boundaries under asymmetric costs. Across all ex-

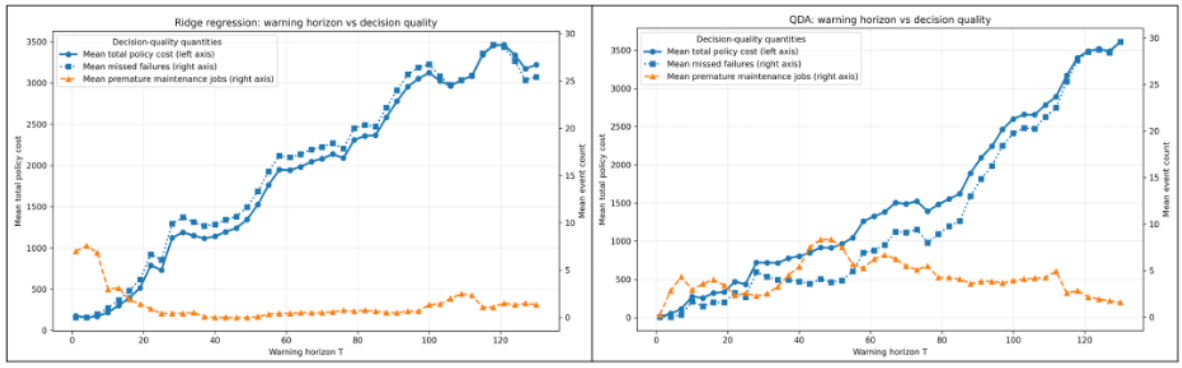

Figure 3. Decision quality over various warning horizons for ridge regression and QDA.

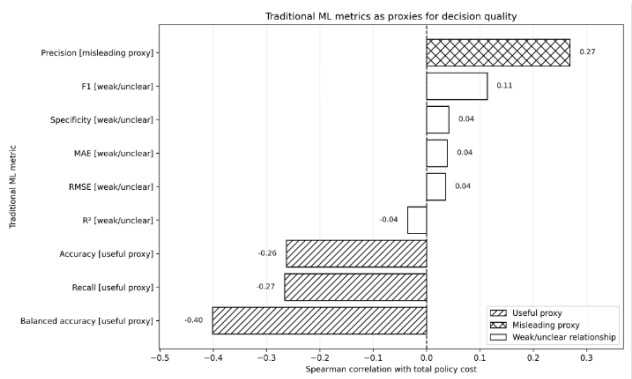

Figure 4. Traditional ML metrics as proxies for decision quality.

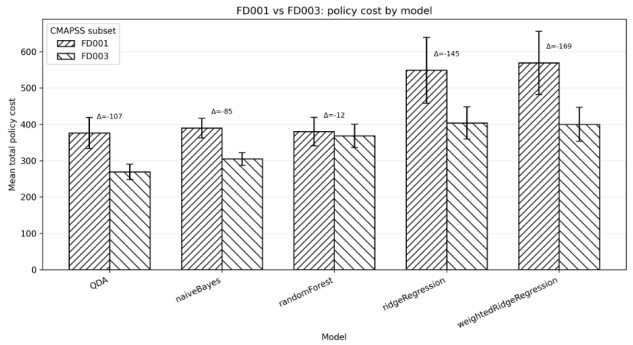

Figure 5. Policy cost by model, FD001 vs. FD003.

periments, improvements in prediction only translate into operational value when they alter decisions; errors that do not change the induced action are inconsequential. Furthermore, performance depends strongly on operating regime, class balance, and the separability of degradation signals rather than on physical system complexity alone.

A key limitation of our work is the one-shot decision formulation. In practice, maintenance decisions are sequential, and optimal policies must account for the evolving state of the system and the option to act over time. As a result, the apparent optimality of short-horizon decisions in our experiments is likely an artifact of the modeling assumption rather than a property of real systems.

These findings suggest that future predictive maintenance systems should move beyond prediction-centric pipelines toward decision-aware and policy-based formulations, where models are trained and evaluated directly with respect to their impact on sequential decision quality.

Description of Individual Effort

The project was structured with a division of responsibilities across modeling, pipeline development, and analysis, with iterative integration throughout.

Cole H. Shaigec was responsible for the overall system architecture, including design of the experimental pipeline, data preprocessing and EDA on the CMAPSS dataset, and implementation of the Naive Bayes and weighted ridge regression models. He also led experiment design, hyperparameter tuning strategy, and end-to-end evaluation, including analysis of both predictive and policy-level performance metrics. In addition, he coordinated integration across model implementations and maintained the shared codebase.

Kelly Falcón focused on implementing tree-based and linear modeling components within the shared framework, including random forest, ridge regression, and PCA-based dimensionality reduction. This included adapting these methods to the project's data structures and ensuring compatibility with the pipeline's training and evaluation interfaces. She also contributed to the literature review by identifying and summarizing relevant prior work.

Youwei Chen focused on probabilistic and instance-based models, implementing QDA and kNN within the shared pipeline. This included developing the underlying mathematical formulation for QDA and ensuring correct integration with the dataset and evaluation framework. He

also contributed to the literature review and supported validation of model behavior during experimentation.

All members contributed to discussion of modeling choices and interpretation of results, particularly in comparing model performance across different operating conditions and decision thresholds.

References

[1] Air Cargo Week, "The true cost of engine maintenance," 2023. [Online]. Available: https://aircargoweek.com/the-true-cost-of-engine-maintenance/

[2] A. Saxena and K. Goebel, "Turbofan Engine Degradation Simulation Data Set," NASA Prognostics Data Repository, 2008.

[3] T. Wang, J. Yu, D. Siegel, and J. Lee, "A similarity-based prognostics approach for remaining useful life estimation," IEEE PHM Conference, 2008.

[4] A. de Giorgio, G. Cola, and L. Wang, "Systematic review of class imbalance problems in manufacturing," Journal of Manufacturing Systems, 2023.

[5] O. Oleghe, "A predictive noise correction methodology for manufacturing process datasets," Journal of Big Data, 2020.

[6] W. B. Powell, *Bridging Decision Problems: Volume 1: Framing the Problem*. Wiley, 2022.

[7] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," Technometrics, 1970.

[8] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, 2009.

[9] L. Breiman, "Random forests," Machine Learning, 2001.

A. Dataset Details and Summary Statistics

The NASA CMAPSS dataset contains simulated turbofan engine degradation trajectories generated from a high-fidelity engine simulation environment. Each row corresponds to one engine-cycle observation and contains an engine identifier, cycle index, operational setting variables, and multivariate sensor readings. The learning target is remaining useful life (RUL), defined as the number of cycles remaining until the simulated engine reaches the terminal degradation condition.

We restrict the study to FD001 and FD003 because both operate under a single operating condition, allowing us to focus on differences in failure-mode structure rather than confounding the study with multiple operating regimes. FD001 contains one degradation mode, high-pressure compressor (HPC) degradation. FD003 contains two interacting degradation modes, HPC and fan degradation. Table 1 summarizes the subsets used in the project.

Table 1. Dataset summary statistics for CMAPSS subsets used in this study.

Quantity	**FD001**	**FD003**
Failure mode(s)	HPC	HPC, fan
Engines (train)	100	100
Engines (test)	100	100
Sensors	21	21
Records (train)	20,631	24,720
Records (test)	13,096	16,596
Mean lifetime (train)	206.31	247.20
Mean lifetime (test)	206.48	241.28
Lifetime SD (train)	46.34	86.48
Lifetime SD (test)	44.04	76.03

The higher lifetime variance in FD003 indicates greater heterogeneity in trajectory lengths and degradation behavior. The test trajectories are right-censored: they stop before terminal failure, and the true RUL at truncation is provided separately. This makes the evaluation problem closer to operational prediction, where complete future degradation trajectories are not observed at decision time.

B. Exploratory Data Analysis and Preprocessing Details

Exploratory data analysis showed that the sensor channels vary substantially in scale and informativeness. Some sensors exhibit visible degradation trends over engine lifetime, while others are nearly constant. The preprocessing pipeline therefore performs feature selection, standardization, optional dimensionality reduction, and sliding-window feature construction.

B.1. Low-Variance Sensor Removal

Sensors with near-zero variance across engine trajectories are removed because they carry little predictive signal and can increase dimensionality without improving learning. This step is unsupervised and does not use RUL labels. The practical purpose is to remove flat or nearly flat sensor channels before windowing expands the feature dimension.

B.2. Z-Score Standardization

Because CMAPSS sensors operate on different physical scales, features are standardized within each cross-validation fold. For training-fold mean μ_j and standard deviation σ_j, feature j is transformed as

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}. \tag{3}$$

The transform is fit on the training fold and applied to validation/test data to avoid leakage.

B.3. Sliding-Window Feature Construction

Given an engine trajectory

$$\mathcal{T}_e = \{\mathbf{s}_{e,1}, \ldots, \mathbf{s}_{e,L_e}\}, \qquad \mathbf{s}_{e,t} \in \mathbb{R}^{d_s}, \tag{4}$$

with lifetime L_e and d_s retained sensors, a window of length w ending at time t is flattened into

$$\mathbf{x}_{e,t}^{(w)} = \begin{bmatrix}\mathbf{s}_{e,t-w+1}^T & \mathbf{s}_{e,t-w+2}^T & \cdots & \mathbf{s}_{e,t}^T\end{bmatrix}^T \in \mathbb{R}^{wd_s}. \tag{5}$$

The corresponding regression label is

$$r_{e,t} = L_e - t. \tag{6}$$

For classification at warning horizon T, the corresponding label is

$$y_{e,t}^{(T)} = \mathrm{sign}(T - r_{e,t}). \tag{7}$$

The key computational implication is dimensionality expansion: increasing w multiplies the feature dimension by w. The key statistical implication is strong local correlation among adjacent samples, because consecutive windows overlap. This motivates engine-level cross-validation rather than random sample-level splitting.

B.4. PCA Non-Result

PCA was evaluated as an optional dimensionality reduction step. Empirically, it did not improve predictive or decision performance and was excluded from final configurations. The likely reason is that PCA preserves variance rather than decision-relevant structure: directions of high variance need not correspond to directions that separate near-failure from far-from-failure samples or improve the induced maintenance policy. In this problem, removing variance without regard to decision boundaries can distort the geometry most relevant to cost-sensitive threshold decisions.

C. Experimental Search Space

The pipeline evaluates a configurable experimental space spanning preprocessing, model choice, warning horizon, window size, and CMAPSS subset. The broad design is summarized in Table 2.

Table 2. High-level experimental search space.

Component	Values / Role
PCA	none; variance threshold; fixed components
Models	QDA; Naive Bayes; weighted ridge; ridge; kNN; RF
Warning horizon T	positive integer sweep
Window size w	positive integer sweep
CMAPSS subset	FD001; FD003

The full Cartesian product is not always run at once; instead, broad sweeps identify promising model families and regimes, followed by narrower targeted experiments. This structure enables large-scale experimentation while preserving a consistent evaluation protocol.

D. QDA Implementation

Quadratic Discriminant Analysis (QDA) is a generative classifier that models each class with its own multivariate Gaussian distribution. Unlike LDA, QDA does not assume a shared covariance matrix across classes, allowing quadratic decision boundaries.

D.1. Data and Notation

Let

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}, \qquad \mathbf{x}_i \in \mathbb{R}^d, \tag{8}$$

with labels $y_i \in \{-1, +1\}$. For class $k \in \{-1, +1\}$, define

$$\mathcal{I}_k := \{i : y_i = k\}, \qquad n_k := |\mathcal{I}_k|. \tag{9}$$

D.2. Class Priors

Class priors are estimated empirically:

$$\hat{\pi}_k = \widehat{\mathbb{P}}(y = k) = \frac{n_k}{n}, \qquad \log \hat{\pi}_k = \log\left(\frac{n_k}{n}\right). \tag{10}$$

D.3. Class Means

The class mean vector is estimated by

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i \in \mathcal{I}_k} \mathbf{x}_i. \tag{11}$$

D.4. Regularized Class Covariance Matrices

The empirical class covariance matrix is

$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{i \in \mathcal{I}_k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T. \tag{12}$$

To improve numerical stability, especially when the feature dimension is large after windowing, we use ridge-style covariance regularization:

$$\hat{\Sigma}_k^{(\mathrm{reg})} = \hat{\Sigma}_k + \lambda I_d, \tag{13}$$

where $\lambda > 0$ is tuned by cross-validation.

D.5. Cholesky-Based Numerical Evaluation

Rather than explicitly computing $\left(\hat{\Sigma}_k^{(\mathrm{reg})}\right)^{-1}$, we use the Cholesky factorization

$$\hat{\Sigma}_k^{(\mathrm{reg})} = R_k^T R_k. \tag{14}$$

This enables stable computation of the Mahalanobis term by solving triangular systems. The log determinant is computed as

$$\log \left|\hat{\Sigma}_k^{(\mathrm{reg})}\right| = 2 \sum_{j=1}^{d} \log(R_{k,jj}). \tag{15}$$

D.6. Quadratic Discriminant Rule

For each sample $\mathbf{x}$, QDA computes

$$\delta_k(\mathbf{x}) = \log \hat{\pi}_k - \frac{1}{2} \log \left|\hat{\Sigma}_k^{(\mathrm{reg})}\right| - \frac{1}{2} (\mathbf{x} - \hat{\boldsymbol{\mu}}_k)^T \left(\hat{\Sigma}_k^{(\mathrm{reg})}\right)^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}_k). \tag{16}$$

The predicted label is

$$\hat{y} = \arg \max_{k \in \{-1,+1\}} \delta_k(\mathbf{x}). \tag{17}$$

D.7. Pseudocode

```
Input: training data X, labels y, regularization lambda
for each class k in {-1,+1}:
    I_k = indices where y == k
    n_k = number of samples in I_k
    log_prior[k] = log(n_k / n)
    mu[k] = mean(X[I_k, :])
    Sigma[k] = covariance(X[I_k, :])
    Sigma_reg[k] = Sigma[k] + lambda * I
    R[k] = chol(Sigma_reg[k])
    logdet[k] = 2 * sum(log(diag(R[k])))

for each sample x:
    for each class k:
        z = solve_triangular(R[k], x - mu[k])
        mahal = z' * z
        score[k] = log_prior[k] - 0.5*logdet[k] - 0.5*mahal
    yhat = class with largest score
```

E. Weighted Ridge Regression Implementation

Weighted ridge regression modifies ordinary ridge regression by increasing the influence of near-failure samples. This is intended as a heuristic proxy for the cost-sensitive objective: errors near the warning boundary are more likely to change maintenance decisions and therefore matter more operationally.

E.1. Weighting Scheme

For sample i with true RUL r_i, define the unnormalized weight

$$a_i := 1 + \eta \exp\left(-\frac{r_i}{\tau}\right), \tag{18}$$

where η controls the magnitude of the near-failure emphasis and τ controls the width of the near-failure region. The normalized weight is

$$\gamma_i := \frac{a_i}{\sum_{k=1}^{n} a_k}. \tag{19}$$

The constant term ensures every sample remains represented; the exponential term is largest when RUL is small and decays as RUL increases.

E.2. Weighted Centering

Let $X \in \mathbb{R}^{n \times d}$ and $\mathbf{y} \in \mathbb{R}^n$. Define

$$\hat{\boldsymbol{\mu}}_x := X^T \boldsymbol{\gamma}, \qquad \hat{\mu}_y := \boldsymbol{\gamma}^T \mathbf{y}. \tag{20}$$

The centered variables are

$$\tilde{X} := X - \mathbf{1}\hat{\boldsymbol{\mu}}_x^T, \qquad \tilde{\mathbf{y}} := \mathbf{y} - \hat{\mu}_y \mathbf{1}. \tag{21}$$

E.3. Weighted Covariance Form

The weighted covariance and cross-covariance matrices are

$$\hat{S}_{x,w} := \tilde{X}^T \operatorname{diag}(\boldsymbol{\gamma}) \tilde{X}, \qquad \hat{S}_{xy,w} := \tilde{X}^T \operatorname{diag}(\boldsymbol{\gamma}) \tilde{\mathbf{y}}. \tag{22}$$

The closed-form weighted ridge coefficient vector is

$$\mathbf{w}_{\text{ridge}} := \left(\hat{S}_{x,w} + \frac{\lambda}{n} I_d\right)^{-1} \hat{S}_{xy,w}. \tag{23}$$

The intercept is

$$b_{\text{ridge}} := \hat{\mu}_y - \hat{\boldsymbol{\mu}}_x^T \mathbf{w}_{\text{ridge}}. \tag{24}$$

Predictions are computed as

$$\hat{\mathbf{y}} := \tilde{X} \mathbf{w}_{\text{ridge}} + b_{\text{ridge}} \mathbf{1}. \tag{25}$$

E.4. Equivalent Optimization View

The same estimator can be viewed as minimizing a weighted penalized least-squares objective:

$$\min_{\mathbf{w}} (\mathbf{y} - X\mathbf{w})^T W (\mathbf{y} - X\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2, \tag{26}$$

where $W = \operatorname{diag}(\boldsymbol{\gamma})$. Setting the gradient to zero yields

$$(X^T W X + \lambda I)\mathbf{w} = X^T W \mathbf{y}, \tag{27}$$

which gives the familiar closed form

$$\mathbf{w} = (X^T W X + \lambda I)^{-1} X^T W \mathbf{y}. \tag{28}$$

E.5. Pseudocode

```
Input: X, y, RUL vector r, lambda, eta, tau
for each sample i:
    a[i] = 1 + eta * exp(-r[i] / tau)
gamma = a / sum(a)

mu_x = X' * gamma
mu_y = gamma' * y
X_tilde = X - ones(n,1) * mu_x'
y_tilde = y - mu_y * ones(n,1)

S_xw  = X_tilde' * diag(gamma) * X_tilde
S_xyw = X_tilde' * diag(gamma) * y_tilde
w = solve(S_xw + (lambda/n)*I, S_xyw)
b = mu_y - mu_x' * w

yhat = X_tilde * w + b
```

F. Naive Bayes Implementation

We implement Gaussian Naive Bayes as a MAP classifier. The model is "naive" because it assumes conditional independence of features given the class label.

F.1. Data and Model Assumptions

Let

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}, \qquad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \{-1, +1\}. \tag{29}$$

For class k, the conditional independence assumption is

$$\mathbb{P}(\mathbf{x} \mid y = k) = \prod_{j=1}^{d} \mathbb{P}(x_j \mid y = k). \tag{30}$$

For Gaussian Naive Bayes,

$$x_j \mid y = k \sim \mathcal{N}(\mu_{k,j}, \sigma_{k,j}^2). \tag{31}$$

F.2. Class Indices and Priors

Define

$$\mathcal{I}_k := \{i : y_i = k\}, \qquad n_k := |\mathcal{I}_k|. \tag{32}$$

The class prior is estimated as

$$\hat{\pi}_k := \widehat{\mathbb{P}}(y = k) = \frac{n_k}{n}. \tag{33}$$

F.3. Class-Conditional Feature Statistics

For each class k and feature j,

$$\hat{\mu}_{k,j} := \frac{1}{n_k} \sum_{i \in \mathcal{I}_k} x_{ij}, \tag{34}$$

$$\hat{\sigma}_{k,j}^2 := \frac{1}{n_k} \sum_{i \in \mathcal{I}_k} (x_{ij} - \hat{\mu}_{k,j})^2. \tag{35}$$

Variance smoothing is applied for numerical stability:

$$\hat{\sigma}_{k,j}^2 \leftarrow \hat{\sigma}_{k,j}^2 + \epsilon. \tag{36}$$

F.4. MAP Prediction Rule

Bayes' rule gives

$$\mathbb{P}(y = k \mid \mathbf{x}) \propto \hat{\pi}_k \prod_{j=1}^{d} \mathbb{P}(x_j \mid y = k). \tag{37}$$

To prevent numerical underflow, we compute log scores:

$$\ell_k(\mathbf{x}) = \log \hat{\pi}_k + \sum_{j=1}^{d} \log \mathcal{N}(x_j \mid \hat{\mu}_{k,j}, \hat{\sigma}^2_{k,j}). \tag{38}$$

Equivalently,

$$\ell_k(\mathbf{x}) = \log \hat{\pi}_k - \frac{1}{2} \sum_{j=1}^{d} \left[\log(2\pi\hat{\sigma}^2_{k,j}) + \frac{(x_j - \hat{\mu}_{k,j})^2}{\hat{\sigma}^2_{k,j}} \right]. \tag{39}$$

The prediction is

$$\hat{y} = \arg\max_{k \in \{-1,+1\}} \ell_k(\mathbf{x}). \tag{40}$$

F.5. Pseudocode

```
Input: training data X, labels y, smoothing epsilon
for each class k in {-1,+1}:
    I_k = indices where y == k
    n_k = number of samples in I_k
    prior[k] = n_k / n
    for each feature j:
        mu[k,j] = mean(X[I_k,j])
        var[k,j] = mean((X[I_k,j] - mu[k,j])^2)
        var[k,j] = var[k,j] + epsilon

for each sample x:
    for each class k:
        score[k] = log(prior[k])
        for each feature j:
            score[k] += log Gaussian(x[j] | mu[k,j], var[k,j])
    yhat = class with largest score
```

G. Weighted Ridge Statistical Tests

Table 3 reports matched comparisons between weighted and unweighted ridge regression over 800 paired configurations. Differences are reported as weighted minus unweighted.

Table 3. Matched weighted/unweighted ridge comparison.

Quantity	**Value**
Mean cost difference (W–UW)	$+0.023$
Cost difference p-value	0.993
Mean RMSE difference	0.00056
RMSE difference p-value	0.982
W–UW missed failures	$+0.0675$
Missed failures p-value	0.011
W–UW premature jobs	-0.219
Premature jobs p-value	4.9×10^{-16}

The table indicates no global improvement in average cost or RMSE. However, weighted ridge reduces premature maintenance actions while increasing missed failures. This supports the main interpretation: weighting changes the location and shape of the learned fit near the decision boundary rather than uniformly improving regression quality.

H. Traditional Metric Correlation Tests

We evaluate traditional metrics as proxies for policy cost using rank correlation analysis. The practical question is whether a metric that is easy to optimize also tracks the actual decision objective.

Table 4. Illustrative Spearman correlations with total policy cost. Negative correlations indicate that higher metric values correspond to lower cost.

Metric	Spearman correlation
Balanced accuracy	-0.40
Recall	-0.27
Accuracy	-0.26
Precision	$+0.27$
F1	$+0.11$
Specificity	$+0.04$
MAE	$+0.04$
RMSE	$+0.04$
R^2	-0.04

The main takeaway is not merely statistical significance, but operational relevance. Classification metrics tied to false negatives, especially recall and balanced accuracy, are more aligned with decision quality because missed failures dominate cost. Regression metrics such as RMSE and MAE may be useful for prediction assessment but are weak proxies for policy value unless residuals shift samples across the maintenance threshold.

I. FD001 vs. FD003 Statistical Tests

Table 5 summarizes the FD001–FD003 cost differences by model. Negative values indicate lower policy cost on FD003.

Table 5. Mean policy cost differences by CMAPSS subset. Differences are FD003 minus FD001.

Model	Difference	Significant?
QDA	-107	yes
Naive Bayes	-85	yes
Random forest	-12	no
Ridge regression	-145	yes
Weighted ridge	-169	yes

The result is counter to the original hypothesis that FD003 would necessarily be harder because it contains interacting failure modes. The empirical result suggests that observable separability in feature space is more important than physical failure-mode count alone.

J. Pipeline Details

The pipeline was designed around standardized interfaces for training, prediction, policy induction, and reporting. Each model consumes a common dataset structure and returns predictions that can be mapped into policy decisions through the same cost model.

J.1. Execution Stages

1. Load CMAPSS subset and raw engine trajectories.
2. Remove low-variance sensors.
3. Partition engines into cross-validation folds.
4. Fit fold-specific preprocessing transforms.

5. Construct sliding-window features.
6. Train model under candidate hyperparameters.
7. Generate predictions on validation or test data.
8. Convert predictions to maintenance policies.
9. Evaluate policy cost and diagnostic metrics.
10. Aggregate results across configurations.

J.2. Pseudocode

```
for each experiment configuration:
    load selected CMAPSS subset
    remove globally low-variance sensors
    split engines into K folds

    for each hyperparameter setting:
        for each CV fold:
            train_engines, val_engines = fold split
            fit preprocessing on train_engines
            transform train_engines and val_engines
            construct sliding-window samples
            train model
            predict validation outputs
            induce maintenance policy
            compute policy cost and ML metrics
        aggregate fold results

    select best hyperparameters
    retrain model on full training set
    evaluate on held-out test set
    write run report and summary table
```

K. Hyperparameter Grids and kNN Removal

The project used grid search for hyperparameter tuning. The principal hyperparameters are shown in Table 6.

Table 6. Model hyperparameters considered in the search grid.

Model	Hyperparameters
QDA	regularization strength λ
Naive Bayes	variance smoothing ϵ
Weighted ridge	τ, η, λ
Unweighted ridge	λ
kNN	k
Random forest	number of trees; min leaf size; predictors sampled

kNN was included in the original design because it provides a simple nonparametric baseline. It was removed from final experiments because sliding-window construction causes dimensionality growth from d_s to wd_s, making repeated distance computation expensive. Since kNN requires distance comparisons at prediction time and across many hyperparameter/window/horizon configurations, it became computationally unattractive relative to the other model families.

L. Limitations and Additional Notes

The main limitation of the study is the one-shot decision framing. Operational maintenance decisions are sequential: delaying maintenance preserves flexibility but increases risk, while early maintenance reduces risk but sacrifices remaining useful life. A fully realistic formulation would require sequential decision modeling, potentially including Markov decision processes, approximate dynamic programming, or other policy-based frameworks.

A second limitation is that the cost model is simplified. The constants C_M, C_F, and α encode maintenance cost, failure cost, and wasted-life penalty, but real airline maintenance decisions would also depend on aircraft schedules, spare engine availability, inspection capacity, crew constraints, regulatory requirements, and network-level disruption costs.